

NETR: A Head-Mounted Gaze Tracking and Reading Assistance System Using ESP32-CAM and PCCR

Maadhav

Indian Institute of Technology Madras

Abstract

We present NETR, a head-mounted eye tracking and gaze-guided reading assistance system built entirely from commodity hardware: two ESP32-CAM modules, two IR LEDs, and a laptop running a Python compute stack. One camera captures the eye; one captures the scene (book or screen). The eye camera detects the pupil and corneal reflections (glints) to compute a Pupil-Cornea Corneal Reflection (PCCR) vector. A polynomial regression model maps PCCR vectors to gaze coordinates in the scene. We document the iterative development of the gaze model — from a single-glint 6-term polynomial (median LOO error 109 px) to a dual-LED SplitGlintModel with Lorentzian leave-one-out (LOO) reweighting achieving **20.8 px** median weighted LOO error. An adaptive calibration system with two-phase point selection (max-min spatial coverage followed by inverse-distance-weighted error-guided spawning) reduces calibration burden. A text line detector using strip projection peak tracking identifies reading lines in the world camera view, and the nearest detected line to the gaze estimate is highlighted in a browser overlay. The complete system runs at 20 fps (eye) and 12 fps (world) with ~ 75 –100 ms end-to-end latency on a mid-range laptop.

1 Introduction

Eye tracking is well-established in research and clinical settings, but commercial systems cost hundreds to thousands of dollars. Webcam-based alternatives [1] require significant computation and are sensitive to head movement. We present NETR (Non-invasive Eye Tracking Rig), a head-mounted system designed around three constraints: (1) *commodity hardware only* — total BOM under \$15; (2) *wireless operation* — both cameras stream over WiFi UDP; and (3) *reading assistance as the application* — detecting which line of text on a physical book a user is currently reading.

The system has two complementary goals. First, *calibrated gaze estimation*: mapping the PCCR signal to scene coordinates via polynomial regression. Second, *text line detection*: locating text lines in the world camera and associating the current gaze with the nearest line.

This report documents the full system design, the iterative development of the gaze model through several failed and successful approaches, and quantitative evaluation on calibration recordings.

2 Hardware Architecture

2.1 Camera Modules

Both cameras are AI-Thinker ESP32-CAM modules (OV2640 sensor, up to 2 MP). They are configured asymmetrically:

- **Eye camera (cam2):** Mounted facing the eye, ~ 3 cm away. Two IR LEDs (850 nm) positioned on either side of the lens provide corneal illumination. Streams on UDP port 5002.
- **World camera (cam1):** Mounted facing forward,

capturing the scene. Streams on UDP port 5000.

The eye camera is physically rotated 90° relative to the world camera. This is compensated in software via the `swap_pccr` flag, which swaps dx/dy axes of the PCCR vector before model inference.

2.2 Firmware

A single firmware image is compiled for both cameras using PlatformIO, with the target selected via build environment flags. The firmware runs eight FreeRTOS tasks distributed across two cores (Table 1).

Table 1: FreeRTOS Task Layout

Task	Core	Purpose
captureTask	0	Camera capture at target FPS
sendTask	1	UDP fragmentation + transmit
cmdTask	1	Receive settings commands
otaTask	1	Over-the-air firmware update
discoveryTask	1	Beacon + laptop IP discovery
timeSyncTask	1	Cristian's algorithm clock sync
ledTask	1	LED state machine
wifiTask	1	WiFi watchdog + reconnect

Camera initialisation always uses `FRAME_SIZE_UXGA` to allocate the maximum DMA buffer, then immediately sets the working resolution. This prevents buffer overflow on runtime resolution changes. After any `set_framesize` call, three frames are flushed to drain stale sensor FIFO data.

2.3 Clock Synchronisation

Both cameras implement Cristian's algorithm for clock synchronisation with a local NTP server on the laptop,

running every 10 seconds. This is critical for correlating eye frames with world frames during fixation aggregation.

2.4 Communication Protocol

Cameras broadcast a discovery beacon on UDP port 5004. The laptop receiver responds with its IP. JPEG frames are fragmented into 1400-byte UDP packets with a 12-byte header (frame ID, fragment index, total, timestamp). The receiver reassembles fragments before decoding.

3 Eye Processing Pipeline

3.1 Pupil Detection

Pupil detection uses a configurable algorithm in `PupilDetector`. The default *threshold* method applies CLAHE contrast enhancement, Gaussian blur, and adaptive thresholding followed by blob detection. The pupil is identified as the largest dark circular region.

3.2 Glint Detection

Corneal reflections from the IR LEDs appear as small bright blobs on the iris. `GlintDetector` operates in two stages:

1. **Limbus estimation:** 16 radial rays from the pupil edge profile intensity outward. The iris-to-sclera brightness jump gives the limbus radius, defining an iris annulus as the search region.
2. **Adaptive thresholding:** Within the iris annulus, the 99th-percentile brightness sets the detection threshold. Blobs passing circularity and area filters are glint candidates. A `min_dist_factor` gate rejects blobs within $0.3\times$ the pupil radius — these are tear film artefacts.

3.3 PCCR Computation

The Pupil-Cornea Corneal Reflection vector is:

$$(dx, dy) = \text{pupil_center} - \text{glint_xy}$$

With `swap_pccr=True` (90° rotated camera):

$$dx_{\text{eff}} \leftarrow dy_{\text{raw}}, \quad dy_{\text{eff}} \leftarrow dx_{\text{raw}}$$

This makes dx the horizontal displacement in real-world space, which correlates strongly with horizontal gaze angle ($r \approx 0.90$). Glints are labeled by LED side using the dy axis (horizontal in real-world space): side = +1 if $dy > \text{switch_dx}$, else -1.

4 Gaze Model: Design Evolution

4.1 Polynomial Basis

All gaze models use a 6-term second-order polynomial:

$$\phi(dx, dy) = [1, dx, dy, dx \cdot dy, dx^2, dy^2]$$

Two independent least-squares regressions map features to screen coordinates:

$$\hat{X} = \phi \mathbf{A}, \quad \hat{Y} = \phi \mathbf{B}, \quad \mathbf{A}, \mathbf{B} \in \mathbb{R}^6$$

4.2 Stage 1 — Single GazeModel (Baseline)

One model trained on the preferred-side glint. Side determined by horizontal sweep calibration: five dots at $y = 50\%$, swept left-to-right, with PCCR crossing point `switch_dx` marking LED visibility change.

Result (28 fixations): LOO median error **109 px**. Corners extrapolate catastrophically (300–500 px).

4.3 Stage 2 — DualGazeModel

Two independent `GazeModel` instances, one per LED side. Both trained by emitting two samples per fixation. At inference, the currently-visible side’s model is used.

Problem: each model sees only half the data; the other LED’s PCCR is discarded at inference time.

4.4 Stage 3 — Rejected Alternatives

We tested three alternative feature representations (Table 2).

Table 2: Feature Comparison (28 fixations, all errors in px)

Method	Mean	Median	p75
Single preferred glint	153	109	163
Midpoint $(g_1 + g_2)/2$	166	112	178
Midpoint $\div D$ (normalised)	171	111	181
4D joint feature	178	150	237

Midpoint/D normalisation: Standard for remote trackers where inter-glint distance D varies with head depth. In our head-mounted rig, LEDs are fixed relative to the eye camera — D varies only 134–140 px across all fixations. Normalising by a near-constant adds no information the polynomial doesn’t already absorb.

4D joint feature: Only 28 samples with cross-terms over 4 input dimensions — catastrophic LOO overfit.

4.5 Stage 4 — TwoGlintGazeModel (11-term)

A single model using both PCCR vectors:

$$\phi_{11} = [1, dx_1, dy_1, dx_2, dy_2, dx_1 dx_2, dx_1 dy_2, dy_1 dx_2, dy_1 dy_2, dx_1^2, dx_2^2]$$

Cross-terms capture the *differential signal* between the two glints — a much stronger predictor than either PCCR alone. R^2 improved from ~ 0.72 to ~ 0.91 .

Result (63 fixations): LOO median **45 px** (from 63 px).

Why replaced: At $N < 50$, the 11-term model overfits. LOO tail reaches $p_{90} = 753$ px on some recordings — removing one sample leaves only 19 points for 11 free parameters.

4.6 Stage 5 — SplitGlintModel

Two independent 6-term models, one per LED:

- `model_right`: trained on (dx_1, dy_1) , right LED
- `model_left`: trained on (dx_2, dy_2) , left LED

Both trained on frames where *both* glints are detected. Pre-training filters:

1. $|dx| < 10$ px: glint near pupil centre — drop
2. Per-glint sigma filter: $|dy| > \mu + 3\sigma$ — drop
3. Collection gate: $|dy_1| > 120$ or $|dy_2| > 120$ — strip to single-glint

4.6.1 Selection Criterion Evolution

We tested four strategies for choosing which model to use at inference:

1. max $|dx|$ — ignores dy
2. max $|dy|$ — ignores dx
3. max magnitude $\sqrt{dx^2 + dy^2}$ — cross-validated on 10 recordings, mixed results
4. **Always-right, fallback-left** — always use `model_right`

Always-right won on 8/10 recordings. Root cause: the left LED model is consistently undertrained ($\sigma = 34.8$ px vs 12.2 px for right). Any strategy that sometimes selects the weaker left model adds noise.

4.7 Stage 6 — Lorentzian LOO Reweighting (Final)

Before fitting, per-sample LOO residuals are computed. Samples with high LOO error receive lower weight:

$$w_i = \frac{1}{1 + \left(\frac{e_i}{e_{\text{median}}}\right)^2}$$

Weighted least squares is applied by scaling design matrix rows by $\sqrt{w_i}$. The Lorentzian kernel has heavier tails than Gaussian reweighting — it does not aggressively penalise moderate errors, only genuine outliers.

Result (88 samples, recording 20260505_040942):

Table 3: LOO Error: Unweighted vs Lorentzian Weighted (px)

	Mean	Median	p75	p90
Unweighted	43.5	28.0	46.5	83.0
Weighted	31.6	20.8	31.6	57.1

67 of 87 samples improved. Mean -27% , p90 -31% .

5 Calibration System

5.1 Fixation Aggregation

During calibration, a 500 ms capture window is opened per target dot. Eye frames are aggregated:

1. **Homography sync:** For each eye frame, the world frame is fetched by timestamp. ArUco markers are detected and a homography H projects the stimulus target (s_x, s_y) into world-camera coordinates (X, Y) — the model training target.

2. **Blink filter:** Frames with pupil radius deviating $> 15\%$ from median are dropped.
3. **IQR filter:** dx and dy filtered by $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$.
4. **Mean:** Mean of surviving (dx, dy, X, Y) becomes the training sample.

5.2 Adaptive Point Spawning

Fixed-grid calibration wastes samples in accurate regions while undersampling high-error corners. We implemented two-phase adaptive selection:

Phase 1 — Bootstrap ($N < 9$): Maximise minimum distance to all collected points — ensures spatial coverage before any error signal exists.

Phase 2 — Error-guided: After each fixation, LOO errors are recomputed. Each candidate is scored by IDW over the error map:

$$\text{score}(c) = \frac{\sum_i e_i / d(c, p_i)^2}{\sum_i 1 / d(c, p_i)^2}$$

The candidate with the highest score is selected next — biasing new samples toward high-error screen regions (typically corners underserved by ArUco markers).

6 Text Line Detection

6.1 Motivation

The target application requires identifying which line of a physical book a user is reading. Challenges: page curvature, uneven illumination, two-page spreads, tilted camera angle.

6.2 Curve Track Algorithm

Rather than detecting individual words, we track the ink *density baseline* of each line across the page width — naturally following curvature.

6.2.1 Preprocessing

$$\text{BGR} \xrightarrow{\text{gray}} \xrightarrow{\text{CLAHE}} \xrightarrow{\text{blur}} \xrightarrow{\text{adaptive threshold}} \text{bin_inv}$$

`bin_inv` is a binary image where ink pixels are white. Adaptive threshold handles uneven illumination across the page; global thresholding would miss text in shadow regions.

6.2.2 Strip Projection

The image is sliced into N vertical strips (default 24). Per strip:

$$\text{profile}[y] = \sum_{x \in \text{strip}} \text{bin_inv}(x, y)$$

Peaks in the profile correspond to text line centres.

6.2.3 Greedy Peak Tracking

Peaks are found by NMS with minimum relative height and minimum inter-peak distance constraints. Tracks extend greedily left-to-right: peaks are assigned to the nearest existing track within y_{tol} pixels. Unmatched peaks start new tracks. Tracks are killed after `track_max_gap` missed strips.

6.2.4 Three-Layer Outlier Rejection

Adaptive threshold converts shadow/hand boundaries into thin horizontal edge artefacts with the same FWHM as real text lines. Three layers address this:

1. **FWHM filter:** Peaks wider than $2.5\times$ global median FWHM are dropped.
2. **Neighbour support:** A peak must have a matching peak in an adjacent strip within y_{tol} — removes single-strip spikes.
3. **Spacing check:** Track pairs closer than 55% of median inter-track gap are shadow edge pairs; the shorter is dropped.

6.2.5 Polygon Output

Each track is a sparse set of (x, y) samples. A degree-2 polynomial is fitted and evaluated densely. The baseline curve is offset $\pm\frac{1}{2}$ inter-line gap to produce a closed curved polygon strip per detected line.

6.3 Book Page Segmentation

To discard detections on the desk, hands, and background, a bright-region page mask is computed: threshold at the 60th brightness percentile, morphological close + open to fill text gaps, then largest-contour convex hull approximated to a quad. Line centres outside the quad are discarded.

6.4 Active Line Detection

The nearest text line to the current gaze is found by minimum 2D segment distance over all track segments. The closest track is highlighted in the browser overlay as the active (currently-read) line.

6.5 Temporal Stabilisation

An optional optical flow stabilisation pass (off by default) warps previous-frame track positions onto the current frame using Lucas-Kanade sparse flow on `goodFeaturesToTrack` keypoints within the page region, followed by RANSAC homography estimation.

7 Receiver and Browser Interface

The Python receiver (`receiver.py`) is the central processing hub: it receives UDP streams from both cameras, runs `EyePipeline` at 20 fps, buffers world frames for homography correlation, and serves a browser dashboard on port 8080.

The browser overlay renders: pupil circle (green), limbus ring (dashed), glint dots (yellow = right LED, blue = left LED), PCCR arrow on the eye view; and on the

world view: dashed blue book quad, dim green text line polygons, bright yellow active line with glow, and a gaze crosshair.

A frame hash cache prevents re-running `detect_lines` when the world frame has not changed between gaze polling cycles (~ 20 Hz poll vs ~ 12 fps world camera).

8 Evaluation

8.1 Gaze Model Accuracy

All evaluation uses LOO cross-validation on two-glint samples. Model targets are world-camera-space coordinates (X, Y) from projecting stimulus positions through the ArUco homography — isolating model accuracy from homography coverage quality.

Best recording (20260505_040942, 76 two-glint samples): LOO model error: median **25.2 px**, p90 82.5 px.

End-to-end error (predicted vs actual stimulus) is ~ 297 px median. This is entirely attributable to ArUco coverage: markers were placed only in the central 60% of the screen. Error by screen X zone confirms this — rising from 120 px (leftmost zone) to 466 px (rightmost zone, $s_x > 1200$ px). Placing markers at screen corners would collapse end-to-end error to approximately the LOO model error.

8.2 Temporal Stability

Within-fixation PCCR jitter: median frame-to-frame displacement 1.6 px, within-fixation standard deviation 2.1 px. The ~ 25 px LOO error is systematic (calibration/homography), not random noise — confirming no temporal smoothing is needed.

8.3 System Latency

Table 4: Latency Breakdown (estimated)

Stage	Latency
Eye camera capture (20 fps)	50 ms
UDP transmission	2–5 ms
Pupil + glint detection	8–15 ms
PCCR + model inference	<1 ms
Browser poll + render	16–33 ms
Total	$\sim 75\text{--}100$ ms

9 Design Decisions and Lessons Learned

Head-mounted vs remote: Head-mounting eliminates head-movement compensation entirely. The PCCR is stable across large head poses because the LED-camera geometry is fixed. The tradeoff is wearability and the 90° rotation requiring `swap_pccr`.

Two cameras vs one: The second (world) camera is essential — the gaze model outputs world-camera coordinates; without it, there is no way to identify which physical object is being looked at.

Two LEDs vs one: A single LED gives one PCCR vector; two LEDs allow SplitGlintModel to use the differential signal, improving R^2 from ~ 0.72 to ~ 0.91 . The decision to use two separate 6-term models instead of one 11-term joint model was driven by sample sparsity — with $N < 100$ typical in a session, the 11-term model overfits severely.

Always-right LED: Empirical cross-validation on 10 recordings showed always using the right LED model outperforms dynamic magnitude-based selection. The asymmetry arises from consistent physical placement: the right LED sits at a slightly better angle relative to the cornea in the head-mounted rig.

Adaptive vs grid calibration: Grid calibration wastes budget on the already-accurate screen centre. Phase 1 (max-min distance) ensures coverage; Phase 2 (IDW error-guided) concentrates the remaining budget on the high-error corner regions — exactly where ArUco coverage is weakest.

Curve track vs MSER: MSER requires global skew estimation, which fails on curved book pages. `curve_track` requires no skew assumption — strip projection peaks naturally follow curvature. Three-layer shadow rejection was necessary in practice: adaptive threshold creates false edges at shadow and hand boundaries with the same FWHM as real text lines.

10 Limitations and Future Work

Homography coverage: The current bottleneck. End-to-end error drops from ~ 300 px to ~ 25 px with full-screen ArUco corner markers.

Session persistence: Gaze model is not persistent across sessions. Drift in head-rig position requires recalibration. A 10–15 point incremental recalibration using the adaptive system could maintain accuracy across sessions.

Left LED model quality: The left LED model has consistently higher variance ($\sigma = 34.8$ px vs 12.2 px). Better LED positioning or higher IR power on the left side would improve the fallback model and potentially enable reliable dynamic selection.

Word-level detection: The system identifies which *line* is being read, not which *word*. Word-level accuracy is limited by the ~ 25 px LOO error (word width ≈ 30 – 60 px at typical reading distance).

Reading speed inference: Future work could use the temporal sequence of active-line transitions to estimate reading speed and potentially identify re-reading (comprehension difficulty).

11 Conclusion

We have presented NETR, a complete head-mounted gaze tracking and reading assistance system built from commodity hardware. The key contributions are:

1. A dual-LED **SplitGlintModel** with Lorentzian LOO reweighting achieving **20.8 px** median weighted LOO error — a $4.8\times$ improvement over the single-glint baseline (109 px).
2. An **adaptive two-phase calibration** system (max-min spatial coverage + IDW error-guided spawning) concentrating sample budget on high-error screen regions.
3. A **strip projection peak tracking** text line detector (`curve_track`) with three-layer shadow rejection, page quad masking, and optional optical flow stabilisation, designed for live book reading.
4. Full **wireless operation** at 20 fps / 12 fps with ~ 75 – 100 ms end-to-end latency on two \$7 ESP32-CAM modules.

The primary remaining bottleneck is homography coverage: ArUco markers at screen corners would reduce end-to-end gaze-to-screen error from ~ 300 px to ~ 25 px, enabling word-level reading detection.

References

- [1] A. Papoutsaki, P. Sangkloy, J. Laskey, N. Daskalova, J. Huang, and J. Heer. WebGazer: Scalable webcam eye tracking using user interactions. In *Proc. IJCAI*, 2016.
- [2] C. H. Morimoto and M. R. M. Mimica. Eye gaze tracking techniques for interactive applications. *Computer Vision and Image Understanding*, 98(1):4–24, 2005.
- [3] J. J. Cerrolaza, A. Villanueva, and R. Cabeza. Taxonomic study of polynomial regressions applied to the calibration of video-oculographic systems. In *Proc. ETRA*, 2012.
- [4] L. R. Young and D. Sheena. Survey of eye movement recording methods. *Behavior Research Methods & Instrumentation*, 7(5):397–429, 1975.